

# Alcançando *Maior Produtividade*

Conhecer os recursos é metade do caminho. A outra metade é o **método**: administrar o contexto, entender antes de mudar, planejar, deixar o agente se verificar com testes, revisar bem, trabalhar em paralelo e automatizar o que se repete, sem perder o controle do custo.

Contexto

Entender → planejar

TDD com agentes

Verificação

Revisão e commits

Paralelo e nuvem

Custo

Formação Claude Code 2026 · Baseado nos guias de [cursor.com/learn](https://cursor.com/learn) (Agents, Context, Understanding your codebase, Developing features, Reviewing and testing code) e na documentação do Cursor (Prompting agents, Subagents, Worktrees, Models & Pricing), consultados em outubro de 2026

## De quem faz *a quem coordena*

O guia do Cursor define um agente como "**ferramentas em um laço**": você dá um objetivo e ele decide os passos. Isso muda o seu papel: de quem executa cada tarefa para quem **gerencia as tarefas**.

### COMO TRATAR O AGENTE

O próprio Cursor compara: um **desenvolvedor júnior muito rápido**, que precisa de direção clara, esquece coisas e pode entrar em looping repetindo a mesma abordagem errada. Ou seja: precisa de supervisão.

### COMO DELEGAR

Comece por **tarefas pequenas e bem definidas** para ganhar confiança. Aos poucos, entregue blocos maiores, sempre com revisão. Você vira o arquiteto e o revisor; o agente cuida da implementação.

Produtividade aqui não é digitar mais rápido: é **ter mais trabalho bem feito acontecendo ao mesmo tempo**, com você nas decisões que importam.

## Trabalhar com IA é *administrar contexto*

Segundo o guia, a qualidade da resposta depende menos do prompt "perfeito" e mais do **contexto** que o modelo recebe: o que está na conversa, os arquivos, a saída do terminal, as rules. E esse espaço é limitado.

### UMA CONVERSA POR TAREFA

Conversas longas acumulam assuntos velhos. Tarefa nova, conversa nova. Trocar de modo também começa um contexto novo.

### CONTEXTO CERTO

Use `@` quando souber os arquivos. Quando não souber, deixe o agente buscar; ele usa o subagente Explore sem lotar a conversa.

### OLHO NO ANEL

O anel ao lado da caixa de mensagem mostra o quanto já foi usado e de quê. Perto do limite, o Cursor resume a conversa; no CLI, `/summarize`.

**Rules enxutas:** toda rule "Always Apply" ocupa contexto em todo pedido. Prefira rules por arquivo ( `globs` ) ou por descrição.

**MCP com moderação:** cada servidor conectado traz instruções e catálogo de ferramentas para o contexto. Ligue só os que você usa.

# Entender *antes de mudar*

O erro mais comum, segundo o guia: pedir mudança sem entender o que já existe. O agente cria uma função que já existia ou usa um padrão diferente do resto do projeto.

Antes de mudar qualquer coisa, me mostre como funciona a validação de formulários hoje. Quais padrões usamos e onde ficam os validadores compartilhados?

Os agentes levam o pedido ao pé da letra. Onde você não diz a intenção, eles decidem por conta própria.

| TIPO DE PERGUNTA              | EXEMPLO                                                                                                        |
|-------------------------------|----------------------------------------------------------------------------------------------------------------|
| Específica (busca exata)      | "Ache todos os arquivos que importam o PaymentService e mostre como tratam o PaymentFailedError."              |
| Ampla (busca por significado) | "Como o app trata pagamentos que falham? Mostre o fluxo do erro, do formulário até a mensagem para o usuário." |
| Visual                        | "Crie um diagrama Mermaid do fluxo de dados do pagamento, do checkout até a integração com o Stripe."          |

Comece **específico** quando sabe o que procura e **amplo** quando está explorando. Diagramas ajudam no onboarding e revelam problemas de arquitetura.

## Planos que o agente *consegue verificar*

A chave, segundo o guia, é **quebrar o trabalho em passos que o próprio agente consiga verificar**. O Plan Mode faz perguntas e gera um plano com tarefas que você edita antes de construir.

```
# preferencias-notificacao.md (resumo do exemplo do guia)
### Abordagem
- Seguir o layout de src/pages/Settings.tsx
- Reusar o componente Toggle da biblioteca de UI
### Tarefas
- [x] Coluna notification_preferences no banco
- [ ] Componente NotificationPreferences
- [ ] Rota app/api/user/notifications/route.ts
- [ ] Atualização otimista com o hook existente
- [ ] Testes do toggle e da validação da API
```

### BOM PLANO

Diz **o que reusar** (arquivos, componentes, padrões), separa tarefas pequenas e já inclui os testes.

### QUANDO RECOMEÇAR

Se o resultado errou o alvo, não remende com dez pedidos. **Desfaça, deixe o plano mais específico e rode de novo**. Parece contraintuitivo, mas costuma ser mais rápido.

A parte difícil quase sempre é decidir **o que fazer**. Com instruções certas, a implementação é a parte fácil de delegar.

## TDD com agentes: *testes primeiro*

O agente rende mais quando consegue saber se acertou. Teste que falha é retorno concreto: ele vê o erro, ajusta e tenta de novo.

**1 Peça os testes primeiro**, com entradas e saídas esperadas. Diga que é TDD, para ele não inventar mocks de código que não existe.

**2 Confirme que falham**. Ainda não é hora de escrever a funcionalidade.

**3 Faça commit dos testes**. Eles viram o contrato.

**4 Peça o código** para passar em todos os testes, **sem alterar os testes**.

**5 Revise e faça commit**.

# Passo 1

Escreva testes para uma função `descontoCupom()` que:

- devolve o preço com desconto para um cupom válido
- lança `CupomInvalidoError` para cupom vencido
- aplica descontos de valor fixo ("100FF" = R\$ 10)
- nunca devolve preço negativo (mínimo R\$ 0)

Siga o padrão de `@pricing.test.ts`. NÃO escreva a função ainda.

# Passo 4

Faça todos os testes de `@descontoCupom.test.ts` passarem, seguindo o padrão de `@PricingService.ts`. NÃO altere os testes.

## Dê ao agente *como se conferir*

O maior risco de construir rápido é **pular a verificação**. Quanto mais checagens automáticas o projeto tiver, mais você pode delegar com segurança.

### TESTES

Pegam regressões de comportamento.

### TIPAGEM

Pega erros de estrutura. O guia recomenda linguagens tipadas.

### LINTER

Pega desvios de estilo e de padrão.

### NAVEGADOR E MCP

Dão retorno visual sobre mudanças de interface.

**Deixe o agente escrever os testes que faltam.** Boas perguntas: "Que casos-limite não estão cobertos nos testes do desconto?", "Escreva testes de regressão para o bug que corrigimos em PaymentService.ts."

**Sem infraestrutura de testes?** O agente pode montar do zero, por exemplo configurar o Playwright e criar os primeiros testes de ponta a ponta.

## Do desenho *ao código*

### MOCKUPS

Cole um wireframe ou uma exportação do Figma na caixa de mensagem e peça o componente.

### BUG VISUAL

Mande um print do estado errado da tela e peça para investigar.

### ITERAÇÃO

Tire um print do resultado atual e descreva o que precisa mudar.

Conecte o **MCP do Figma** e o agente lê tokens de design, variáveis e especificações dos componentes direto dos seus arquivos.

Com o **navegador integrado**, o agente abre a página, tira capturas e confere o próprio resultado visual antes de devolver para você.

O guia avisa: acertar o layout **pixel por pixel** ainda é um ponto fraco dos agentes. Espere chegar perto e ajuste nos detalhes.

## Revise antes de *pedir revisão*

Código gerado por IA pode parecer certo e estar sutilmente errado: segue o padrão, compila, passa nos testes, mas esquece um caso-limite, abre uma falha de segurança ou duplica lógica. O padrão de qualidade do merge deve ser o mesmo, venha o código de quem vier.

- ✓ **Acompanhe o agente trabalhando.** Indo para o lado errado? *Stop* ( `Ctrl` + `Shift` + `Backspace` ) e redirecione
- ✓ **Revise tudo de uma vez** com `@Branch` : o diff inteiro do branch
- ✓ **Review** → **Find Issues** depois da tarefa, ou Agent Review na aba Source Control
- ✓ **Bugbot** no PR, com autofix para aplicar a correção pelo comentário

Revise as mudanças deste branch (@Branch). Procure bugs, tratamento de erro faltando e o que não segue o padrão de @PricingService.ts.

Que perguntas um revisor vai fazer sobre estas mudanças? O que devo explicar na descrição do PR?

## Commits pequenos, *sem esforço manual*

O agente produz muita mudança de uma vez: um commit com centenas de linhas é difícil de revisar. O guia recomenda **commits pequenos e semânticos**, cada um com uma mudança lógica. E sugere deixar o agente fazer essa arrumação:

- 1 Construa à vontade, sem se preocupar com os commits.
- 2 Quando tudo funcionar, peça ao agente para reorganizar o histórico em partes revisáveis.
- 3 Ele volta à `main`, lê as mudanças e cria commits lógicos com mensagens claras.
- 4 No fim, confere que o diff final é igual ao original: nada se perde.

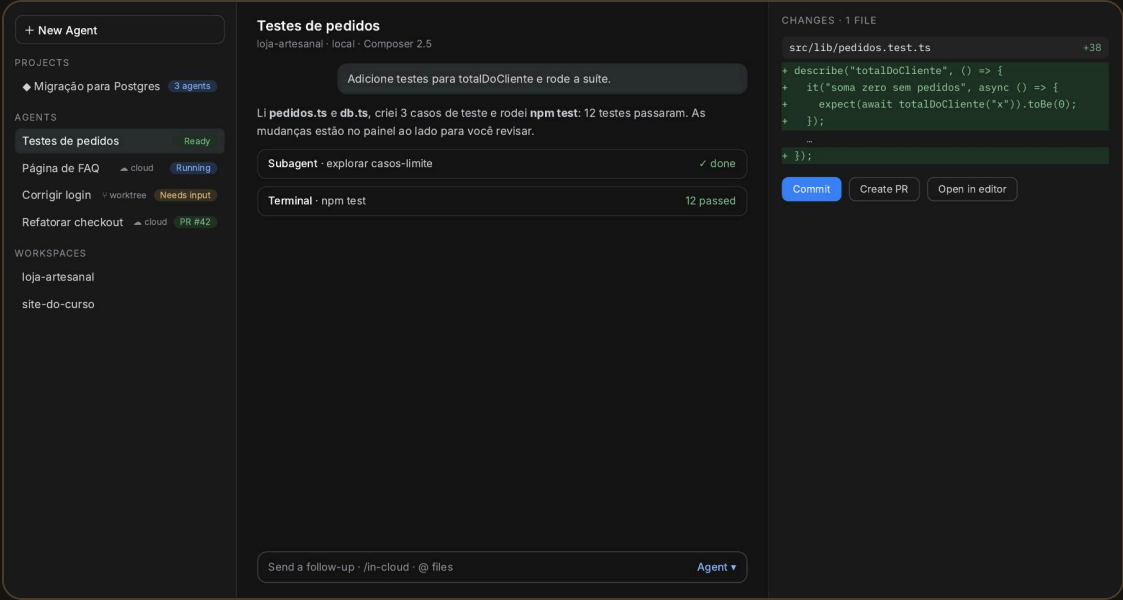
### VIRE UMA SKILL

Transforme esse pedido numa skill, como `/rework-commits`, e qualquer pessoa da equipe roda no fim de uma funcionalidade.

É a mesma ideia de "um commit por aula" que usamos neste curso: cada commit conta uma parte da história.

Reescrever histórico é seguro **antes** de publicar o branch. Depois do push, combine com a equipe.

# Várias frentes *ao mesmo tempo*



Agentes locais, em worktree e na nuvem lado a lado · reprodução ilustrativa

### WORKTREES

Cada agente numa cópia isolada do repositório. Vários agentes no mesmo projeto, sem conflito.

### A LISTA DE PENDÊNCIAS VAI PARA A NUVEM

Bug que apareceu no caminho, teste que falta, documentação, refatoração: Cloud Agent, que abre o PR e avisa quando termina.

### TESTAR EM ESCALA

Vários Cloud Agents testando, em paralelo, casos-limite, erros e combinações de entrada diferentes.

# Automatize *o que se repete*

| VOCÊ SE PEGA...                             | TRANSFORME EM                         | EXEMPLO                                                                |
|---------------------------------------------|---------------------------------------|------------------------------------------------------------------------|
| Repetindo a mesma orientação em todo pedido | Rule ( <code>/create-rule</code> )    | "Use pnpm", "componentes com menos de 200 linhas"                      |
| Escrevendo o mesmo roteiro de trabalho      | Skill ou Custom Mode                  | <code>/rework-commits</code> , checklist de revisão, <code>/tdd</code> |
| Corrigindo a formatação depois do agente    | Hook <code>afterFileEdit</code>       | Rodar o formatador a cada edição                                       |
| Fazendo a mesma checagem toda semana        | Automation ( <code>/automate</code> ) | Revisar os PRs da semana e mandar resumo no Slack                      |
| Acompanhando um PR até ficar verde          | <code>/autopilot</code>               | Responder comentários e corrigir checks                                |
| Perseguindo um objetivo longo               | <code>/goal</code>                    | "Deixar o CI verde"                                                    |

Versione rules, skills, hooks e subagentes do projeto junto com o código. O que um aprende, a equipe inteira passa a usar.

# Um fluxo *sem tirar a mão do teclado*

| PARA                               | FAÇA                                                                         |
|------------------------------------|------------------------------------------------------------------------------|
| Escrever o óbvio mais rápido       | <code>Tab</code> aceita; <code>Tab</code> de novo pula para a próxima edição |
| Ajustar um trecho                  | Selecione + <code>Ctrl</code> + <code>K</code>                               |
| Levar a seleção para o agente      | <code>Ctrl</code> + <code>L</code>                                           |
| Trocar de modo / de modelo         | <code>Shift</code> + <code>Tab</code> / <code>Ctrl</code> + <code>/</code>   |
| Já deixar o próximo pedido na fila | <code>Enter</code> enquanto o agente trabalha                                |
| Corrigir o rumo agora              | <code>Ctrl</code> + <code>Enter</code>                                       |
| Tirar uma dúvida sem atrapalhar    | <code>/btw</code> abre uma conversa lateral                                  |
| Achar uma conversa antiga          | <code>Ctrl</code> + <code>K</code> na Agents Window                          |

No Mac, troque `Ctrl` por `Cmd`. Você também pode **ditar** o pedido pelo microfone da caixa de mensagem.

# Produtivo *sem estourar o plano*

| PERFIL (SEGUNDO O CURSOR)   | GASTO TÍPICO                       |
|-----------------------------|------------------------------------|
| Usa o Tab todo dia          | Dentro do incluído                 |
| Usa o agente às vezes       | Costuma ficar dentro do incluído   |
| Usa o agente todo dia       | US\$ 60 a 100/mês no total         |
| Vários agentes e automações | Muitas vezes acima de US\$ 200/mês |

Agentes gastam bem mais tokens que perguntas simples, por causa das chamadas de ferramenta e das tentativas.

- ✓ Modelo rápido e barato (ex.: Composer 2.5) para o rotineiro; o mais capaz só onde precisa
- ✓ Ou deixe no **Auto**
- ✓ Conversas curtas e focadas consomem menos contexto
- ✓ Acompanhe em `cursor.com/dashboard/usage` os dois bolsos de uso
- ✓ Uso sob demanda é cobrado ao preço de API: defina um teto antes de ligar
- ✓ Automações e Cloud Agents também consomem. Revise as que rodam sozinhas

## Onde o agente *brilha e onde tropeça*

### VAI MUITO BEM

- ✓ Tarefas com objetivo claro e padrões já existentes
- ✓ Adicionar testes a código existente
- ✓ Atualizar documentação
- ✓ Refatorar seguindo um padrão consistente
- ✓ Corrigir bugs com mensagem de erro clara

### AINDA TROPEÇA

- Depuração que exige entender interações complexas do sistema (aqui, use o Debug Mode)
- Reproduzir um layout pixel por pixel
- Bibliotecas muito novas, fora do que o modelo conhece
- Ficar em looping na mesma abordagem que falha
- Fazer, com entusiasmo, mudanças que você não pediu

E o mais importante, nas palavras do guia: **você continua responsável** pelo código que vai para produção.

# Um dia *produtivo com o Cursor*

| MOMENTO          | O QUE VOCÊ FAZ                                                   | RECURSO                                                |
|------------------|------------------------------------------------------------------|--------------------------------------------------------|
| Começo           | Lê o resumo da automação da noite e os PRs que o Bugbot comentou | Automations, Bugbot                                    |
| Tarefa principal | Pede um mapa da área do código, planeja e aprova o plano         | Ask, Plan Mode                                         |
| Implementação    | Testes primeiro, commit, depois o código até tudo passar         | Agent, TDD                                             |
| Em paralelo      | Manda o bug que apareceu no caminho para a nuvem                 | Cloud Agent, <code>/in-cloud</code>                    |
| Ajustes finos    | Pequenas mudanças direto no editor                               | Tab, Ctrl+K                                            |
| Antes do PR      | Revisão do branch, histórico em commits pequenos                 | <code>@Branch</code> , Agent Review, skill de commits  |
| PR aberto        | Deixa o agente cuidar dos comentários e checks                   | <code>/autopilot</code> , Bugbot                       |
| Fim do dia       | Registra o que aprendeu numa rule ou skill                       | <code>/create-rule</code> , <code>/create-skill</code> |

## Para levar *para o seu dia a dia*

- ✓ Uma conversa por tarefa
- ✓ Explorar antes de mudar
- ✓ Plan Mode para o que toca vários arquivos
- ✓ Testes primeiro, sem deixar o agente alterá-los
- ✓ Testes, tipos e linter para o agente se verificar
- ✓ Revisar o branch inteiro antes do PR
- ✓ Commits pequenos e semânticos
- ✓ Pendências paralelas na nuvem ou em worktrees
- ✓ O repetido vira rule, skill, hook ou automação
- ✓ Modelo do tamanho da tarefa
- ✓ Olho no anel de contexto e no painel de uso
- ✓ Errou feio? Desfaça, refine e recomece

## Teste o que *ficou*

1. No TDD com agentes, o que dizer ao pedir o código?

A) Pode mudar os testes se precisar

B) Faça todos os testes passarem sem alterar os testes

C) Apague os testes que falharem

D) Não rode os testes

**B.** Os testes são o contrato.

2. O resultado do plano errou o alvo. O que o guia recomenda?

A) Remendar com vários pedidos

B) Trocar de editor

C) Desfazer, deixar o plano mais específico e rodar de novo

D) Aceitar assim mesmo

**C.** Costuma ser mais rápido.

3. Como pedir ao agente a revisão de todas as mudanças do branch?

A) `@Branch` no pedido

B) `@Terminals`

C) `/goal`

D) Ctrl+K

**A.** Traz o diff inteiro contra a main.

4. Qual a melhor forma de lidar com uma orientação que você repete em todo pedido?

A) Continuar repetindo

B) Transformar numa rule

C) Trocar de modelo

D) Desligar o Tab

**B.** `/create-rule` ajuda a criar.

# Você no comando, *agentes no trabalho*

Neste módulo você viu **o que é o Cursor** (aula 1), **como usar** no dia a dia (aula 2), **as principais funcionalidades** (aula 3) e, agora, **o método** que transforma tudo isso em produtividade: administrar o contexto, entender antes de mudar, planejar em passos verificáveis, testar primeiro, revisar bem, fazer commits pequenos, trabalhar em paralelo, automatizar o repetitivo e escolher o modelo pelo tamanho da tarefa.

Contexto

Explorar → planejar

TDD

Testes · tipos · linter

@Branch · Bugbot

Worktrees · nuvem

Rules · skills · hooks

Fim do módulo 34 · Introdução ao Cursor

Fontes: [cursor.com/learn](https://cursor.com/learn) (Agents, Context, Understanding your codebase, Developing features, Reviewing and testing code) e documentação do Cursor (Prompting agents, Subagents, Worktrees, Agent Review, Bugbot, Automations, Models & Pricing), consultados em outubro de 2026. Formação Claude Code 2026.